HYPER-LOCAL AVIATION WEATHER FORECASTING AND VERIFICATION SYSTEM

INDIVIDUAL THESIS

H.M.S.I. Jayawardhana

(IT22619976)

Individual component: Cloud and Visibility Prediction Model

Group ID: R26-IT-132

Report submitted in partial fulfilment of the requirements for the bachelor's degree thesis

Sri Lanka Institute of Information Technology

Department of Computing

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. Any use of generative AI or automated tools must be disclosed in accordance with the module policy. The candidate remains responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis. This draft must be reviewed and corrected by the candidate before submission.

Student name: H.M.S.I. Jayawardhana

Student ID: IT22619976

Signature: _____ Date: 2026/10/10

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	**Signature**	**Date**
Supervisor: Ms. Shashika Lokuliyana		
Co-supervisor: Ms. Hansika Mahaadikara		

ABSTRACT

This thesis documents the individual research and development work on a Cloud and Visibility Prediction Model within the Hyper-Local Aviation Weather Forecasting and Verification System. The component investigates how historical aviation weather observations and associated meteorological variables can be prepared and used to estimate cloud and visibility conditions in a local airport context. It is presented as a research prototype and is not a replacement for official meteorological judgement or an operationally certified aviation system. The work follows the documented path from weather observations and METAR parsing through data preparation, feature representation, model development, evaluation, integration, and presentation of prediction outputs. The supplied research paper reports accuracy values of 76.25% for Decision Tree, 76.31% for Random Forest, and 78.15% for XGBoost, and identifies XGBoost as the selected approach. The available implementation scripts use RandomForestClassifier, so the paper's XGBoost result cannot be assumed to be reproducible from those scripts without further evidence. This thesis keeps reported results separate from implementation details that remain to be verified. It considers data quality, target representation, temporal validation, evaluation measures, system integration, responsible use, and the limitations of the available evidence. The contribution is a documented component design and a testable evaluation workflow for cloud and visibility prediction using local aviation weather data. Further controlled testing, traceable experiment records, and validation against additional observed cases are required before any operational use.

Keywords: aviation weather, METAR, cloud prediction, visibility prediction, machine learning, model validation

ACKNOWLEDGEMENTS

I would like to express my sincere appreciation to my supervisor and co-supervisor for their guidance, constructive feedback, and academic support throughout this research project. I also wish to thank Mr. Ajith Wijemanna, Director General of the Department of Meteorology, and Mr. Jeewan Karunaratna, Head Forecaster at the Department of Meteorology, for their support and domain guidance. I acknowledge the support provided by the project team and the technical resources used to develop and evaluate the forecasting models. I am grateful to all individuals and institutions who supported this research.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

ABSTRACT ii

ACKNOWLEDGEMENTS iii

LIST OF FIGURES x

LIST OF TABLES xi

LIST OF ABBREVIATIONS xii

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 1

1.3 Problem statement 2

1.4 Research gap 3

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 4

1.7 Scope, limitations and assumptions 4

1.8 Individual contribution 5

1.9 Thesis structure 5

2 LITERATURE REVIEW AND RELATED WORK 6

2.1 Review method and scope 6

2.2 Domain and theoretical foundations 6

2.3 Related research 7

2.4 Existing systems, products and techniques 8

2.5 Gap synthesis and conceptual framework 9

3 RESEARCH METHODOLOGY 11

3.1 Research design 11

3.2 Data and study materials 12
3.3 Experimental procedure 12
3.4 Measures and evaluation metrics 13
3.5 Data analysis 14
3.6 Ethics, privacy, security and responsible AI 14
3.7 Reproducibility and research artefacts 15
4 PRODUCT AND SOLUTION DESIGN 17
4.1 Requirements 17
4.2 Component architecture within the complete solution 18
4.3 Design alternatives and decisions 19
4.4 Detailed design 20
4.5 Standards and professional practice 21
5 IMPLEMENTATION, INTEGRATION AND TESTING 23
5.1 Development environment and technology stack 23
5.2 Component implementation 23
5.3 Integration with other components 24
5.4 Verification and software testing 25
5.5 Deployment and operation 26
6 RESULTS AND VALIDATION 28
6.1 Research results 28
6.2 Product and technical results 29
6.3 User or stakeholder validation 30
6.4 Integration results 30
6.5 Summary of findings 31
7 DISCUSSION 32
7.1 Interpretation of findings 32

7.2 Comparison with literature and existing solutions 32
7.3 Research contribution 33
7.4 Product contribution 33
7.5 Limitations and threats to validity 33
7.6 Implications 34
8 CONCLUSION AND FUTURE DIRECTIONS 35
8.1 Conclusion 35
8.2 Achievement of objectives 35
8.3 Individual research and product contributions 35
8.4 Future directions 36
REFERENCES 37
References 38
GLOSSARY 40
APPENDICES 42
Appendix A. Extended technical details and workflow 42
A.1 Feature documentation checklist 43
Appendix A.2 Data quality and preprocessing protocol 45
A.2.1 Raw-data preservation 45
A.2.2 Timestamp normalisation 45
A.2.3 Duplicate observations 45
A.2.4 Missing and invalid values 45
A.2.5 Outlier handling 46
Appendix A.3 METAR parsing and target-definition considerations 47
A.3.1 Parsing is part of the model 47
A.3.2 Visibility target representation 47
A.3.3 Cloud target representation 47

A.3.4 Present-weather encoding 47
A.3.5 Target leakage examples 48
Appendix A.4 Model training and validation protocol 49
A.4.1 Establishing a baseline 49
A.4.2 Chronological partitioning 49
A.4.3 Hyperparameter selection 49
A.4.4 Class imbalance 49
A.4.5 Repeated runs and uncertainty 50
Appendix A.5 Interpreting cloud and visibility metrics 51
A.5.1 Accuracy and its limits 51
A.5.2 Mean absolute error 51
A.5.3 Root mean squared error 51
A.5.5 Confusion matrix and event-level analysis 51
Appendix A.6 API contract and error-handling specification 53
A.6.1 Request contract 53
A.6.2 Response contract 53
A.6.3 Error responses 53
A.6.4 Observability 53
A.6.5 Versioning 53
Appendix A.7 Dashboard presentation and accessibility 55
A.7.1 Distinguishing observation and prediction 55
A.7.2 Displaying units and rounding 55
A.7.4 Accessibility 55
A.7.5 User evaluation 55
Appendix A.8 Risk register 56
Appendix A.9 Requirements traceability matrix 58

Appendix A.10 Literature comparison matrix 59
Appendix A.10 Data governance and model card 61
A.10.1 Intended use..... 61
A.10.2 Out-of-scope use 61
A.10.3 Data description..... 61
A.10.4 Model details 61
A.10.5 Performance and limitations 61
A.10.6 Maintenance 62
Appendix B. Test cases and evidence checklist..... 63
Appendix B.2 Academic integrity and AI-use disclosure..... 65

LIST OF FIGURES

Figure 1: Position of the individual component in the complete system 2
Figure 2: Research workflow for the Cloud and Visibility Prediction Model 11
Figure 3: Logical component architecture 19
Figure 4: Data-to-prediction flow 21
Figure 5: Inference and dashboard integration flow 24
Figure 6: Evaluation workflow 28

LIST OF TABLES

Table 1: Literature themes relevant to cloud and visibility prediction 10
Table 2: Meteorological fields and feature documentation 16
Table 3: Functional requirements for the prediction component 18
Table 4: Non-functional requirements and evidence 22
Table 5: Verification test cases for parsing and prediction 26
Table 6: Model performance values reported in the group research paper 29
Table 7: Data-to-prediction processing stages 43
Table 8: Evaluation metrics and their limitations 52
Table 9: Risk register for the prediction component 57
Table 10: Requirements traceability matrix 58
Table 11: Literature comparison matrix 60
Table 12: Model-card fields and current evidence status 62
Table 13: Extended parser and integration test cases 65

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
ICAO	International Civil Aviation Organization
MAE	Mean Absolute Error
METAR	Meteorological Aerodrome Report
ML	Machine Learning
RF	Random Forest
RMSE	Root Mean Squared Error
TAF	Terminal Aerodrome Forecast
UTC	Coordinated Universal Time
VCBI	Bandaranaike International Airport location indicator
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall project context

Aviation weather information supports decisions made before and during aircraft operations. Conditions such as low cloud, fog, and reduced visibility can influence whether an approach can be conducted, whether a runway can be used under the applicable procedures, and whether operations require additional caution. A weather report is therefore not simply a description of the atmosphere; it is an input to a safety-critical decision process. The accuracy and timeliness of the information matter, especially when conditions change over a short period.

The group research project proposes a hyper-local aviation weather forecasting and verification system for Bandaranaike International Airport (VCBI). The project paper describes several distinct forecasting components covering temperature and pressure, wind, cloud and visibility, and QNH/dew point/relative humidity. These are individual sub-systems within a shared workflow that collects meteorological observations, prepares features, generates forecasts, compares forecasts with later observations, and presents relevant information to users. This thesis concentrates on the cloud-and-visibility component rather than claiming responsibility for every group subsystem.

Airport-level forecasting has a different emphasis from a broad regional forecast. A regional product can describe the general weather situation while local conditions near an aerodrome can still change because of moisture, cloud-base variation, rainfall, and other atmospheric processes. The proposed component uses the historical and current METAR information available to the group system to estimate cloud status and visibility for the local operational context. It is intended as supplementary decision support and must not replace official aviation-weather products or authorised operational decisions.

1.2 Individual component context

The group paper describes two separate XGBoost models, one for cloud status and one for visibility. Current METAR data are retrieved, parsed into fields, stored in Superbase according to the paper, transformed through feature engineering, and passed

to the models. A Flask API controls the cloud-and-visibility prediction workflow, and a dashboard presents the resulting information. These descriptions provide the documented design basis for this thesis. The precise source code, model artefacts, database schema, label definitions, and API routes should be included as appendices or evidence in the final version when available.

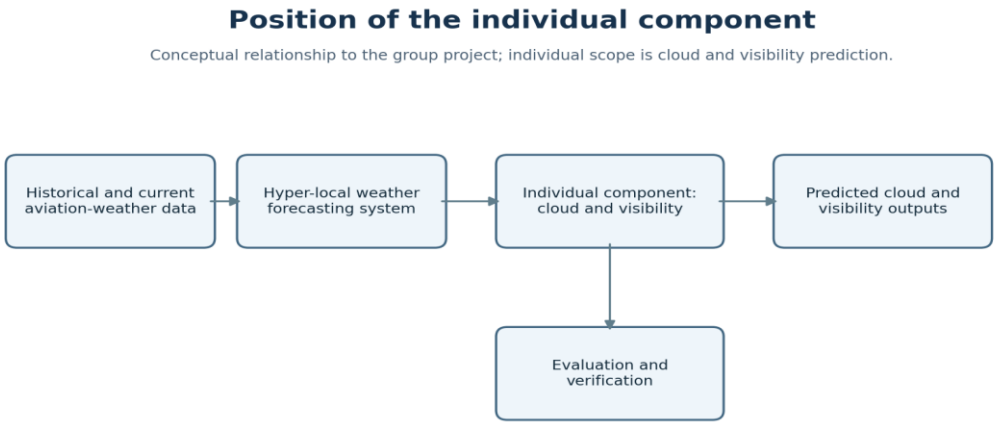


Figure 1: Position of the individual component in the complete system

1.3 Problem statement

Visibility and cloud conditions can change quickly, and a single observation may not adequately describe what will occur during a later flight-related time window. The operational challenge addressed by this component is to use local meteorological observations to forecast cloud status and visibility, while detecting patterns that may indicate reduced visual range or low cloud. Without a locally evaluated prediction component, the wider system cannot provide its intended additional view of these variables alongside its other weather forecasts.

The project assessment form specifically identifies cloud-ceiling analysis, visibility reduction, sudden fog onset, low-level clouds, and rapid atmospheric shifts as the problem area. These phenomena create a modelling challenge because the same input variable may relate differently to visibility under different combinations of temperature, dew point, humidity, wind, pressure, and present weather. Furthermore,

the records available in METAR reports are observations at reporting times and may not capture every short-lived transition between reports. A model must therefore be evaluated for both average performance and its behaviour around important low-visibility cases.

1.4 Research gap

Prior work cited by the project documents includes research on machine-learning-based short-term weather prediction in Sri Lanka, artificial neural networks for rainfall forecasting, studies of aviation weather hazards, and work on automated TAF generation. These studies establish the relevance of machine learning to weather prediction and the importance of aviation-specific meteorological information. However, the group paper argues that many local studies focus on general weather or rainfall prediction rather than a combined, airport-level cloud and visibility workflow with forecast verification.

The gap should not be overstated. XGBoost, feature engineering, and METAR parsing are established techniques. The defensible contribution is their adaptation and integration for the stated VCBI cloud-and-visibility use case, with an explicit evaluation process and an aviation-oriented presentation of predicted conditions. Any claim of novelty beyond this scope requires a systematic comparison with existing research and operational products.

1.5 Aim and objectives

The aim of the individual component is to design, implement, and evaluate a cloud-and-visibility prediction workflow using airport-level METAR observations, and to integrate the resulting outputs into the wider hyper-local aviation weather system for supplementary awareness of reduced-visibility and low-cloud conditions.

Objective 1 is to identify the METAR fields and derived features relevant to cloud status and visibility from the documented project data flow. Objective 2 is to implement or document separate prediction paths for cloud status and visibility using the selected XGBoost approach. Objective 3 is to apply consistent preprocessing between training and inference so that the feature vector supplied to the model has the same schema and semantics as the vector used during training. Objective 4 is to expose

predictions through the component API and make them available to the wider dashboard. Objective 5 is to evaluate model behaviour using appropriate target-specific metrics and a time-aware validation design. Objective 6 is to define and test prediction-output consistency, using documented target definitions rather than arbitrary boundaries.

1.6 Research questions or hypotheses

These questions separate predictive performance from system integration. A model can have acceptable average metrics yet fail to detect rare but important low-visibility events. Conversely, a downstream rule can be implemented correctly but still be unreliable if the underlying prediction is poorly calibrated. Both dimensions need evidence. The questions are therefore intended to guide evaluation rather than imply that the answers are already established.

If hypotheses are required by the department, the principal testable hypothesis may be stated as follows: the selected model provides measurable predictive information for cloud status and visibility beyond a simple persistence baseline on a chronologically separated test set. This hypothesis should be accepted or rejected only after baseline metrics and model metrics have been calculated on the same test observations.

1.7 Scope, limitations and assumptions

The component scope covers METAR-based cloud and visibility prediction, feature preparation, XGBoost inference, API integration, and prediction-output presentation. The research context is VCBI and the project form identifies the First Inter-Monsoon as the component's specialised period. The wider project covers multiple weather variables and seasonal contexts, but those other model components are outside the main technical evaluation of this thesis.

The draft assumes that historical METAR observations and the described feature-generation process are available to the implementation. It does not assume that every proposed input source in the group-level assessment form was ultimately implemented. In particular, the available group paper describes the cloud-and-visibility path in terms of METAR data, parsing, Superbase storage, engineered

features, XGBoost, a Flask API, and a dashboard. Other group-level technologies are mentioned only to explain integration context.

Important limitations include the possible sparsity of low-visibility events, reporting intervals that may miss rapid transitions, potential missing or malformed METAR fields, class imbalance for cloud categories, distribution changes between seasons, and uncertainty about the exact prediction horizon. The model is not certified for operational aviation use. No unreported user study, deployment outcome, threshold source, or model calibration result is claimed in this draft.

1.8 Individual contribution

A defensible contribution statement should be supported by version history, source-code files, commits, test records, model artefacts, and screenshots of the component running. Where a shared team member implemented an API, database, or dashboard, this thesis should acknowledge the integration dependency rather than claim sole ownership. The final version should include a concise table mapping each objective to the code module, test evidence, and result that demonstrates completion.

1.9 Thesis structure

Chapter 1 introduces the project context, individual component, problem, gap, aim, questions, scope, and contribution. Chapter 2 reviews aviation weather, visibility and cloud observations, machine learning for weather, XGBoost, evaluation, and relevant research. Chapter 3 explains the research design, data, preprocessing, modelling procedure, evaluation metrics, responsible use, and reproducibility. Chapter 4 specifies functional and non-functional requirements, architecture, design alternatives, and detailed workflows. Chapter 5 describes implementation and integration. Chapter 6 reports the results that can be substantiated from the available evidence and identifies missing experiment records. Chapter 7 discusses implications, limitations, and contributions. Chapter 8 considers sustainability and responsible deployment. Chapter 9 concludes and recommends future work. References and appendices provide the cited sources and technical evidence.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review scope is defined by the individual component rather than by the entire aviation weather project. Relevant topics include METAR and TAF as aviation-weather products, cloud amount and ceiling reporting, visibility observation and prediction, fog and low-cloud processes, machine-learning approaches for short-term weather prediction, tree-based ensemble methods, and forecast verification. The supplied topic assessment form and group paper provide the initial reference set; those sources are used as project evidence, not as proof that every cited paper directly evaluates the proposed component.

A transparent review should record the search terms, databases, inclusion criteria, and date of the search. A suitable search strategy for completing this thesis would combine terms such as “aviation visibility prediction”, “METAR machine learning”, “fog forecasting airport”, “cloud ceiling prediction”, “XGBoost weather forecasting”, and “automated TAF generation”. Include peer-reviewed studies and official standards or guidance where possible; exclude sources that do not disclose methods or are not relevant to airport-level prediction. Because a full systematic review log was not supplied, this draft describes the review scope but does not claim a completed systematic review.

The comparison should focus on data, forecast horizon, target representation, algorithm, evaluation design, and operational relevance. These dimensions help prevent superficial comparisons between studies that use different airports, forecast horizons, target definitions, or metrics. Numerical performance values should only be compared when the target and evaluation protocol are sufficiently similar.

2.2 Domain and theoretical foundations

METAR is a coded aerodrome weather observation used in aviation. Depending on the report, it can include wind, prevailing visibility, present weather, cloud layers, temperature, dew point, and pressure-related information. These fields are not all continuous numeric measurements. Some are categorical codes, some are numeric values, and some have special encodings for exceptional conditions. A reliable

preprocessing pipeline must parse the format according to the data source and preserve the meaning of each field.

Visibility describes the distance at which objects can be seen and recognised under the applicable observation convention. In an aviation context, visibility and runway visual range are related but distinct concepts; the model should not silently treat them as interchangeable. Cloud information can include cloud amount, layer height, and significant cloud types. A ceiling is generally associated with the lowest broken or overcast layer or vertical visibility under the applicable reporting convention, but the exact interpretation must follow the standard used by the project dataset. These definitions matter because an incorrect target transformation can produce apparently good metrics that do not represent the intended aviation variable.

Fog and low cloud can be associated with high moisture availability and limited mixing, but the atmospheric relationship is not deterministic. Temperature-dew-point spread, relative humidity, wind, pressure tendency, recent weather, and time-of-day patterns can provide predictive signals. However, the same correlation may not hold in every situation, and features derived from variables that are unavailable at the forecast issue time would introduce leakage. Feature engineering must therefore respect the temporal boundary between information known at prediction time and the future observation being predicted.

2.3 Related research

The project topic assessment cites research on artificial neural networks for monsoon rainfall prediction in Sri Lanka and machine-learning-based short-term weather prediction. These works provide a local motivation for applying data-driven methods to meteorological observations. They are not direct substitutes for a cloud-and-visibility study because rainfall targets, spatial scales, forecast horizons, and evaluation metrics can differ substantially from aerodrome visibility or cloud categories.

The project documents also cite work on aviation weather hazards and high-impact weather, including research highlighting the operational importance of thunderstorms, fog, and other adverse conditions. Such literature establishes the need for reliable,

timely information, but hazard studies do not automatically demonstrate the predictive accuracy of a specific XGBoost model. A thesis should use these sources to justify the importance of the problem and use the component's own evaluation to assess the solution.

The group paper discusses automated TAF generation and deep-learning methods for weather nowcasting. These approaches show that automated forecasting can be designed for aviation or for short-term atmospheric evolution. The present component differs in its documented focus on METAR-derived cloud status and visibility features, two XGBoost models, and an API-backed dashboard path. A complete comparison should avoid claiming superiority unless the same test data and target definitions are used.

2.4 Existing systems, products and techniques

Official aviation products remain the reference context for the proposed component. METAR communicates observed conditions and TAF communicates forecast conditions over a defined period. The proposed model is supplementary: it can add a locally learned estimate and verification record, but it cannot replace the official products, published procedures, or operational judgement. The system must label predicted values clearly and distinguish them from observed or officially forecast values.

Numerical weather prediction systems such as the Global Forecast System and ECMWF products provide large-scale guidance based on atmospheric modelling. Their spatial and temporal resolution and data assimilation characteristics differ from a model trained on airport-level observations. A local statistical model may exploit patterns in the local record but may also be less robust when conditions move outside the training distribution. The design therefore treats local prediction as complementary rather than assuming that one approach is universally better.

Among machine-learning techniques, decision trees are interpretable but can be unstable, random forests combine many trees to reduce variance, and gradient-boosted trees iteratively improve a sequence of weak learners. XGBoost is a gradient-boosting implementation that supports regularisation and efficient tree construction. It is

suitable for tabular engineered features, including nonlinear interactions. However, suitability does not guarantee good performance: the data representation, target definition, hyperparameters, class balance, and validation design remain decisive.

2.5 Gap synthesis and conceptual framework

The reviewed material points to a practical gap between general weather prediction and a component designed around airport-level cloud and visibility estimates with a clear prediction and verification pathway. The project contribution is the combination of METAR parsing, feature engineering, separate model targets, API-based inference, and presentation of predicted conditions in the wider system. The component should be assessed for predictive performance, temporal validity, software reliability, and interpretability of prediction outputs.

The conceptual framework follows an input-process-output-feedback sequence. Inputs are historical and current METAR-derived observations. Processing validates and parses records, creates time-aware features, and applies the same transformation used in training. The model produces cloud-status and visibility estimates. A rule layer compares the estimates with configured thresholds. The API exposes results to the dashboard. Once the relevant future observation becomes available, the system can record forecast error and support later evaluation or retraining.

This framework separates model output from decision logic. It also makes the system easier to test: parsing can be validated independently, feature generation can be tested against known records, the model can be evaluated offline, the target interpretation rule can be unit-tested, and the API can be tested with representative payloads. The framework is conceptual until verified against the actual implementation files.

Literature theme	Relevance to this component	Evidence needed for a rigorous comparison

Sri Lankan short-term weather prediction	Supports local motivation for data-driven forecasting	Target, station, forecast horizon, algorithm and reported metrics
Aviation weather hazards	Justifies attention to fog, low cloud and visibility	Operational definitions and relevance to VCBI
Automated TAF generation	Provides aviation-specific automation context	Whether it predicts the same target and horizon
Tree-based ML and XGBoost	Supports selection of a tabular nonlinear model	Baseline comparison, tuning protocol and held-out test results
Forecast verification	Ensures estimates are compared with later observations	Timestamp matching rules and target-specific metrics

Table 1: Literature themes relevant to cloud and visibility prediction

3 RESEARCH METHODOLOGY

3.1 Research design

This work is an applied, quantitative software-and-model research study. It combines the development of a prediction component with empirical evaluation on meteorological observations. The research design should test the model on data that were not used to fit its parameters and should preserve the temporal order of observations where future forecasting is the intended task. A random split may cause records from closely related times to appear in both training and testing sets and can overstate generalisation when weather observations are autocorrelated.

The project paper states that historical VCBI meteorological data from 2019–2026 were used across the wider project and that the data were chronologically ordered and split without shuffling in the general preprocessing description. This is an appropriate starting point for a forecasting evaluation, but the final component-specific thesis must confirm whether the cloud-and-visibility model followed the same split and which dates belong to each partition.

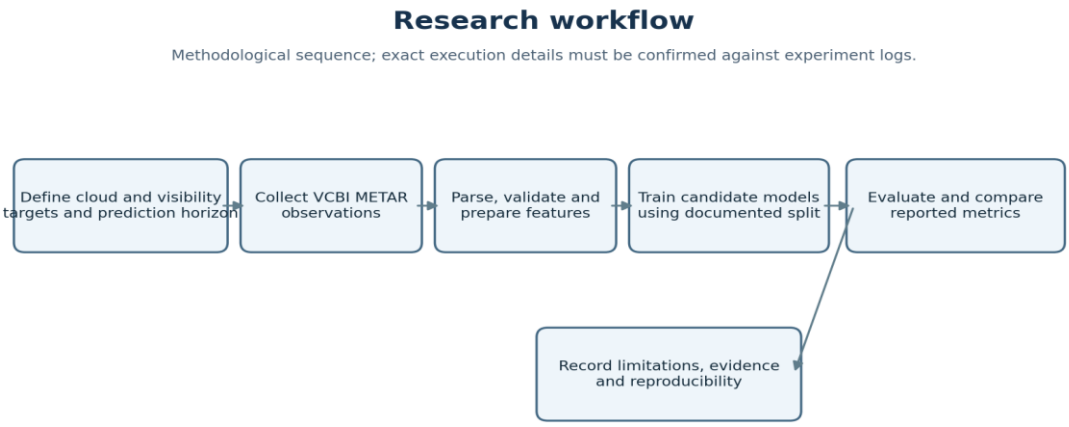


Figure 2: Research workflow for the Cloud and Visibility Prediction Model

The unit of analysis should be one METAR-derived observation associated with an issue timestamp and a defined target timestamp. If the model predicts the next report, the target horizon must be stated in elapsed time rather than assumed from report

frequency. If it predicts a fixed future horizon, the labels must be aligned to that horizon. These are different tasks and should not be combined in one reported score.

3.2 Data and study materials

The principal source described for the component is METAR data for VCBI obtained through the Aviation Weather Center API. The project paper states that parsed real-time records are stored in Superbase and that model features include month, wind direction, wind speed, dry-bulb temperature, dew point, relative humidity, QNH, and present weather. These fields provide context for cloud and visibility prediction; the precise columns used by each model must be confirmed from the actual feature list.

The wider project documentation reports historical meteorological observations spanning 2019–2026 and more than 120,000 samples before cleaning across the project. That total must not be reported as the cloud-and-visibility model's final training size without a component-specific count. Each target can have different missingness, parsing rules, and valid sample counts. The final thesis should report raw rows, duplicates removed, invalid records, missing target values, final rows, and class distribution separately for each model.

METAR strings require careful handling. Visibility may be expressed in different formats, and cloud layers may contain codes and heights rather than simple numeric fields. Present-weather codes can represent fog, mist, rain, or other phenomena. The parser should retain both the original raw report and parsed fields for traceability. A data dictionary should describe the unit, type, valid range, missing-value convention, and transformation for each input and target.

3.3 Experimental procedure

The documented workflow begins with data acquisition and parsing. Each report should be assigned a timestamp, checked for basic validity, and transformed into a structured record. Duplicate timestamps or duplicate reports should be handled according to an explicit rule. Missing values should not be replaced indiscriminately: a missing visibility observation is not equivalent to zero visibility, and a missing cloud layer is not automatically the same as a clear sky unless the METAR encoding confirms that interpretation.

Feature engineering in the group paper includes dew-point depression, temperature-relative-humidity interaction, wind-speed-relative-humidity interaction, QNH-wind interaction, relative humidity squared, and present-weather encoding. The final feature set for each target should be documented independently. Time-derived features may represent the month or hour, but any cyclic encoding should be described with the formula and its period. All transformations must be fitted or specified using training data only where appropriate and then reused unchanged for validation, testing, and inference.

After preprocessing, a baseline should be established. For visibility, persistence or a simple training-set predictor provides a useful reference; for cloud status, a majority-class or persistence baseline can reveal whether the model improves beyond class frequency. The selected XGBoost model should then be trained and evaluated with the same data partitions. If hyperparameters are tuned, the tuning process should use a validation set or time-aware cross-validation rather than the final test set. Once selected, the model and preprocessing artefacts should be versioned together.

3.4 Measures and evaluation metrics

Evaluation must reflect the target type. If visibility is predicted as a continuous value, mean absolute error (MAE) and root mean squared error (RMSE) are relevant. MAE gives the average absolute difference in the target's units, while RMSE penalises larger errors more strongly. If visibility is instead converted into classes, classification metrics such as precision, recall, F1-score, balanced accuracy, and a confusion matrix are required. The final thesis must state which formulation was implemented; it should not report regression error as though it were classification accuracy.

Evaluation timestamps must be aligned so that every prediction is compared with the observation representing its intended target time. Reports with invalid timestamps or ambiguous target matches should be excluded according to a documented rule. Results should include the test period, sample count, missingness, baseline, metric definitions, and uncertainty where feasible. Metrics should not be compared across studies unless units, targets, and evaluation protocols are compatible.

3.5 Data analysis

Exploratory analysis should describe the distribution of visibility and cloud-related labels over time. Useful summaries include counts by month, hour, present-weather code, cloud-status category, and visibility band. Time-series plots can reveal gaps and changes in reporting frequency. Histograms can show skewness and outliers. Cross-tabulations can establish how many fog or low-visibility records exist and whether the data are sufficiently balanced for reliable model training.

Relationship analysis can examine visibility alongside temperature-dew-point spread, relative humidity, wind, pressure, and present weather. Such associations are descriptive and do not establish causation. Correlation can be weak even when nonlinear thresholds exist, and features may be redundant. Feature importance from a tree model can help describe how the fitted model uses inputs, but it should not be interpreted as a causal explanation or as proof that a feature is meteorologically decisive.

Errors should be analysed by condition rather than only in aggregate. For example, the test report should distinguish ordinary visibility cases from fog or low-visibility cases, and clear-sky records from records containing low cloud. Error distributions can show whether the model systematically overestimates or underestimates visibility. If the available evidence does not contain these subgroup results, the thesis should list them as required future evaluation rather than inventing them.

3.6 Ethics, privacy, security and responsible AI

The project assessment form states that the study relies on meteorological and environmental data rather than human participants or personal information. On that basis, human-subject ethical clearance may not be required, subject to the institution's rules. Data source terms, attribution requirements, API conditions, and any restrictions on redistribution should still be respected. The thesis should not publish credentials, API keys, or private configuration values.

Security practices should include secret management, access control for database writes, input validation at API boundaries, and logging that avoids exposing credentials. Model files and preprocessing artefacts should be versioned. Any

deployment must make clear that official METAR/TAF information and approved operational procedures remain authoritative.

3.7 Reproducibility and research artefacts

Reproducibility requires more than saving the fitted XGBoost model. The artefacts should include the raw or appropriately governed source-data reference, cleaning script, parser version, feature-generation code, target construction logic, split timestamps, model configuration, dependency versions, and evaluation script. A fixed random seed may help reproduce training where the algorithm uses randomness, but it does not compensate for changing source data or a different split.

The thesis should identify the version of the model used to generate the reported result and record a checksum or version identifier for the saved artefact. Training and inference must use the same feature names, order, types, missing-value handling, and category encoding. A schema mismatch should produce a clear error rather than a silent incorrect prediction.

The final submission should attach or reference representative code extracts, screenshots, and test logs. Large raw data files do not need to be embedded in the thesis if their size or licensing makes that inappropriate; a controlled repository reference and data dictionary may be more suitable. The candidate should verify that every referenced file is actually present and accessible to the examiner.

Data field or feature described in project paper	**Likely role in workflow**	**Documentation needed**
Month / time	Temporal context	Timezone, encoding, forecast timestamp
Wind direction and speed	Atmospheric context and interactions	Units, calm/variable wind treatment

Dry-bulb temperature	Atmospheric feature	Units and missing-value rule
Dew point	Moisture-related feature	Units and validity checks
Relative humidity	Moisture-related feature	Observed or calculated; range checks
QNH	Pressure context	Units and parsing rule
Present weather	Categorical context	Code mapping and unknown-category handling
Cloud status / cloud layer	Potential target or input	Exact target definition and coding
Visibility	Potential regression or classification target	Units, special encodings, target horizon

Table 2: Meteorological fields and feature documentation

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The component must accept structured meteorological records that have been parsed from METAR observations. It must validate required fields, preserve timestamp information, create the expected features, invoke the correct trained model, and return a structured prediction response. It must handle missing or malformed data explicitly. The API should not return a normal prediction response when required inputs are absent unless a documented fallback policy exists.

ID	Requirement	Priority	Verification method
FR-01	Accept parsed METAR-derived input with timestamp	Must	API schema test
FR-02	Validate required fields and value ranges	Must	Valid/invalid payload tests
FR-03	Apply training-consistent feature transformations	Must	Feature parity test
FR-04	Generate cloud-status prediction	Must	Model inference test
FR-05	Generate visibility prediction	Must	Model inference test
FR-07	Return structured response with timestamps	Must	API contract test

FR-08	Expose results to dashboard integration	Should	Integration test
FR-09	Record prediction and later verification result	Should	Database/log review
FR-10	Report missing/stale input clearly	Must	Failure-path test

Table 3: Functional requirements for the prediction component

The component should produce separate outputs for cloud status and visibility because the group paper describes separate models. The response should identify the model version, issue timestamp, target timestamp or horizon, output value, and any quality or warning status. If a target interpretation rule is triggered, the output should indicate the relevant threshold configuration. These metadata are important for auditability and for matching forecasts to later METAR observations.

The component must also be maintainable. Model artefacts and feature transformations should be versioned; logs should support troubleshooting; and the API should be testable without depending on live data. The final requirements should be validated against the implemented code and any supervisor-approved specifications.

4.2 Component architecture within the complete solution

The cloud-and-visibility component sits between the meteorological data layer and the user-facing presentation layer. At the input side, METAR reports are acquired and parsed. The storage layer described in the group paper uses Superbase for the component's current records. A feature preparation step transforms the parsed fields into the schema expected by the trained XGBoost models. The inference service returns cloud and visibility estimates. A threshold layer evaluates whether the output

meets configured warning conditions, and the dashboard displays the result alongside the wider system's other meteorological outputs.

The group paper describes a Flask API for controlling the cloud-and-visibility prediction process and a Streamlit interface for displaying its information, while the wider project uses a React dashboard and FastAPI services for other system capabilities. The final implementation must clarify the actual boundary between these services. This thesis does not assume that Flask and FastAPI are interchangeable or that every dashboard consumes every endpoint. The architecture should show only the services that are present in the final code.

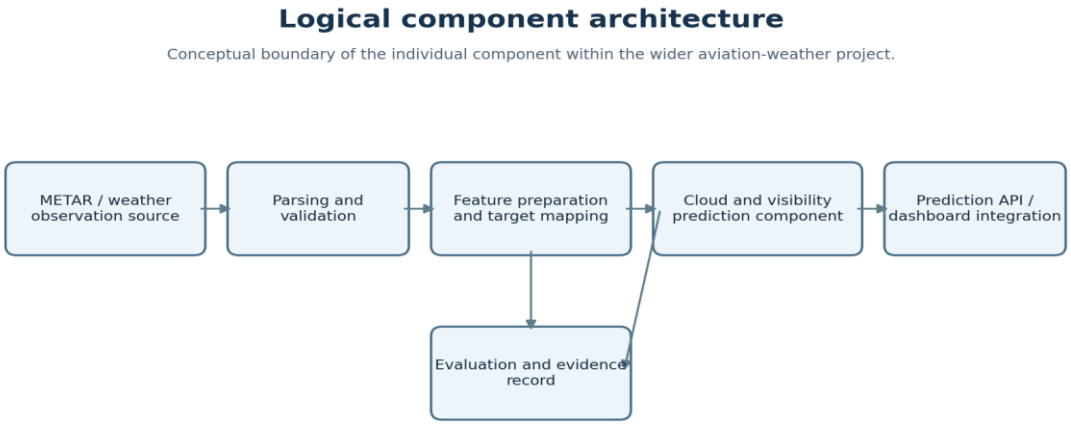


Figure 3: Logical component architecture

4.3 Design alternatives and decisions

A persistence baseline is simple and often difficult to beat over very short horizons, so it should be used as a reference even if it is not the selected production model. Logistic regression or linear regression can provide a transparent baseline depending on target type, while decision trees and random forests offer nonlinear alternatives. The project paper reports a comparison of Decision Tree, Random Forest, and XGBoost for cloud and visibility and states that XGBoost was selected based on the reported results.

XGBoost is a plausible choice for engineered tabular data because it can learn nonlinear interactions and does not require the input to be represented as a long sequential tensor. It can work with mixed feature designs once categorical variables are encoded appropriately. The trade-offs include sensitivity to data leakage, the need for careful tuning, limited extrapolation beyond observed training ranges, and the possibility that aggregate metrics obscure rare adverse events.

Cloud status and visibility are modelled separately in the described system. This design allows each target to have its own label construction, loss function, and evaluation. It also means the outputs may not be physically consistent with one another unless consistency rules are introduced. A future design could compare a multi-task model or a probabilistic model, but changing architecture is not justified without an evaluation demonstrating a benefit.

4.4 Detailed design

The input contract should define each field's name, type, unit, and missing-value behaviour. The parser should normalise the source report into a stable schema and retain the raw report for traceability. Derived features should be generated by a single reusable function shared by training and inference. This prevents the common failure in which the training script and API calculate a feature differently.

The model interface should be deliberately small. It receives a feature vector in the expected order and returns a prediction with model metadata. For a categorical cloud target, the model wrapper may return a class label and, if supported, class probabilities. For a numeric visibility target, it returns the estimate in a documented unit. The final implementation must state which target representation is used; this draft does not infer it from the word “prediction” alone.

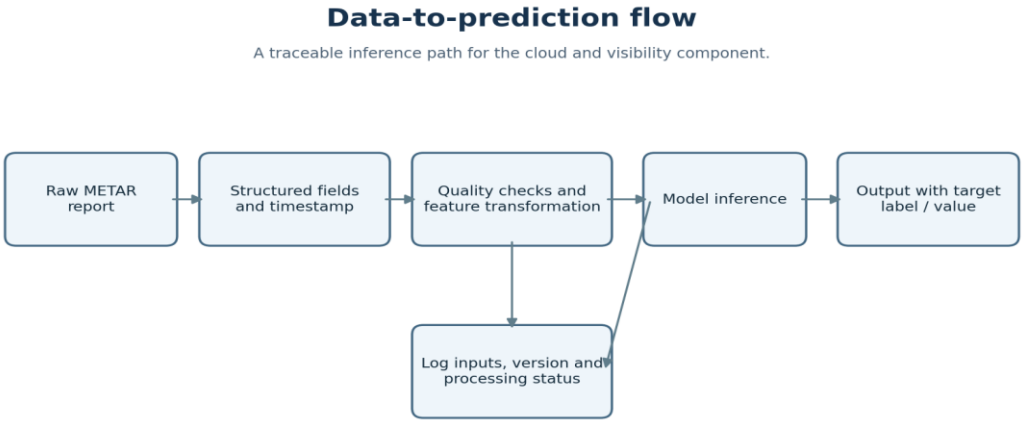


Figure 4: Data-to-prediction flow

4.5 Standards and professional practice

Software engineering practice includes separation of concerns, meaningful error messages, version control, peer review, dependency management, automated tests, and reproducible builds. Data-science practice adds leakage prevention, temporal validation, baseline comparisons, transparent metric reporting, and preservation of test-set integrity. These practices are particularly important because a weather prediction can look plausible even when the input parser has interpreted a coded value incorrectly.

Professional practice also requires clear ownership boundaries in a group project. The thesis should identify the individual component, shared infrastructure, and dependencies on teammates' work. Diagrams and code extracts should be labelled, and external sources should be cited. Any AI assistance should be disclosed according to institutional policy and all generated text and technical claims should be verified by the candidate.

Quality attribute	Design implication	Evidence to collect
Correctness	Stable parsing and consistent feature transformation	Unit tests and known-record checks
Reliability	Explicit handling of missing, stale, or invalid inputs	Failure-path logs
Traceability	Store issue time, target time, model version, and output	Prediction log sample
Maintainability	Separate parsing, features, model, threshold, and API logic	Repository structure
Security	Protect API keys and restrict database writes	Configuration review
Usability	Differentiate predictions, observations, and alerts visually	Dashboard screenshots and review notes

Table 4: Non-functional requirements and evidence

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development environment and technology stack

The group paper identifies Python for model implementation and describes XGBoost for cloud and visibility prediction. It also identifies a Flask API controlling inference and a Streamlit interface for presenting cloud-and-visibility information. Superbase is described as the storage destination for parsed METAR observations. These are the documented technologies for the component; exact package versions, operating system, deployment platform, and repository layout were not included in the supplied documents and should be added from the actual development environment.

A reproducible environment should pin the versions of Python and the libraries used to parse METAR, transform data, load model artefacts, and serve the API. Dependency versions can affect parsing behaviour, model loading, and response serialization. The final thesis should include the actual versions from the environment file or package listing rather than approximate version numbers.

Secrets should be supplied through environment variables or a secure configuration mechanism. Database URLs, API keys, and credentials should never be committed into public source code or included in screenshots. Logs should be useful for debugging while avoiding disclosure of sensitive configuration values.

5.2 Component implementation

The described inference workflow starts when current METAR information becomes available. The record is parsed into fields including time-related context, wind, temperature, dew point, relative humidity, QNH, and present weather. The feature-generation stage derives variables that encode interactions or atmospheric relationships. The same transformations must be applied during training and live prediction, including category handling and missing-value rules.

The project paper reports two separate models for cloud status and visibility. The implementation should load the appropriate artefact, validate its expected feature schema, generate the prediction, and return a response that can be interpreted by the caller. The model-loading process should fail safely if an artefact is missing or

incompatible. A response should not silently reuse an old prediction as if it were current.

5.3 Integration with other components

The wider system combines outputs from multiple weather models and presents them through services and dashboards. The cloud-and-visibility component therefore needs a stable contract that allows other components to consume the predictions without needing to know internal model details. At minimum, the contract should identify the variable, value or category, units, issue timestamp, target timestamp, model version, and warning status.

The group paper describes FastAPI for broader backend capabilities and Flask for the cloud-and-visibility prediction path. If both services remain in the final architecture, the integration boundary should be explicit: one service may call another over HTTP or the cloud-and-visibility service may be consumed independently by the relevant interface. The final thesis should include the real endpoint paths and payload examples from the implementation rather than invented routes.

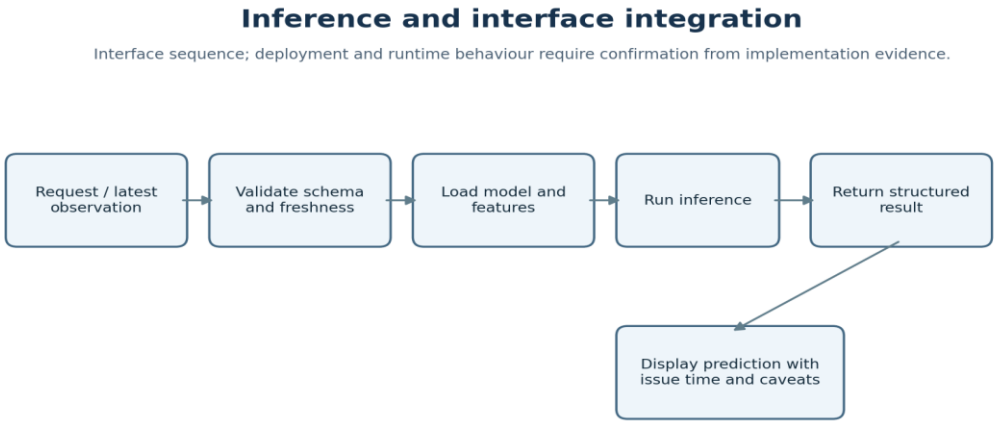


Figure 5: Inference and dashboard integration flow

Integration tests should verify successful requests, invalid inputs, unavailable model files, storage errors, and stale data. They should also verify that the dashboard labels

predicted visibility separately from current observed visibility. A timestamp mismatch can be as damaging as a numerical model error if a forecast is shown as current weather.

5.4 Verification and software testing

Testing should cover unit, integration, model, and end-to-end levels. Unit tests validate parsing, feature generation, unit conversions, and output validation checks. Model tests validate the input schema and output type. API tests verify request validation, response structure, status codes, and error handling. End-to-end tests follow a representative METAR record from ingestion through prediction and presentation.

Test ID	Scenario	Expected behaviour	Evidence status
T01	Valid METAR-derived feature record	Return prediction response with timestamps	To be executed/documented
T02	Missing required visibility feature	Return validation error or documented fallback	To be executed/documented
T03	Unknown present-weather category	Apply defined unknown-category handling	To be executed/documented
T04	Cloud model artefact unavailable	Return controlled service error; no fabricated output	To be executed/documented
T05	Visibility exactly at configured limit	Apply documented boundary rule consistently	Threshold value/rule to confirm

T07	Stale input timestamp	Flag stale input or reject according to policy	To be executed/documented
T08	Database unavailable	Return controlled error and log failure	To be executed/documented
T09	Prediction compared with future METAR	Match forecast and observation timestamps correctly	To be executed/documented
T10	Dashboard receives valid response	Show value, units, timestamp, and warning state	Screenshot/evidence needed

Table 5: Verification test cases for parsing and prediction

Model verification should compare predictions with observations at the intended target time. The test data must not be used to choose hyperparameters or class boundaries. Results should be recorded with the model version and test-period identifiers. A passing software test does not prove predictive accuracy, and a good model score does not prove that the API correctly delivers the result; these are separate claims requiring separate evidence.

5.5 Deployment and operation

The project paper describes a workflow that receives current METAR observations, stores them, prepares features, runs XGBoost inference, and presents cloud and visibility results through an interface. An operational deployment needs health checks, logging, configuration management, and a defined response to data-source outages. If new observations stop arriving, the system should display the last update time and indicate that the information may be stale.

Model updates should be treated as controlled releases. A retrained model should be evaluated against the previous version on a time-separated test set and should only replace the active artefact after required checks pass. The component-specific paper describes retraining based on collected observations in the broader MLOps context, but the exact cadence and deployment mechanism for this component should be confirmed from code or operational logs.

Monitoring should cover both software behaviour and data/model behaviour. Software monitoring includes API errors, latency, failed parsing, and storage availability. Data monitoring includes missing fields, unusual distributions, and changes in class frequency. Model monitoring includes errors once future observations become available and performance by relevant weather condition. A model can continue responding successfully while its predictive quality deteriorates, so both forms of monitoring are necessary.

6 RESULTS AND VALIDATION

6.1 Research results

The supplied group paper reports a comparison among Decision Tree, Random Forest, and XGBoost for the cloud-and-visibility module. It reports accuracy values of 76.25% for Decision Tree, 76.31% for Random Forest, and 78.15% for XGBoost, and gives corresponding error values of 485.1699, 482.9094, and 347.7295. The paper describes XGBoost as the selected algorithm because of the reported comparison. These figures are reproduced as reported by the source, not independently recalculated in this draft.

The interpretation of those numbers requires clarification before final submission. The paper calls the values “accuracy” and “MAE” in the same model comparison, but a value of 347.7295 is not interpretable without knowing the target, units, and scaling. If the target is visibility in metres, the value may represent a substantial error; if the target was scaled or encoded differently, the interpretation changes. It is also unclear from the supplied paper whether the same metric and target representation apply to both cloud status and visibility. The final thesis should inspect the original evaluation script and report target-specific results separately.

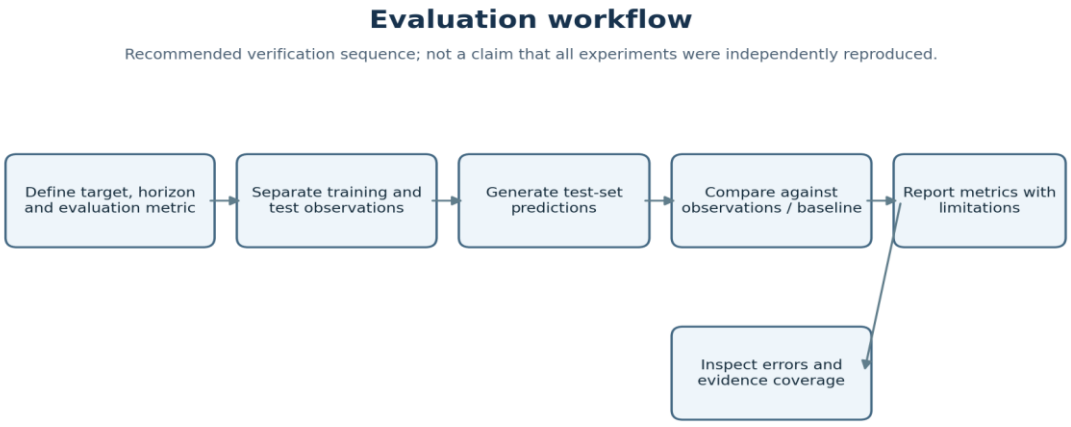


Figure 6: Evaluation workflow

Model as reported in the group paper	Reported accuracy	Reported MAE/error value	Interpretation caveat
Decision Tree	76.25%	485.1699	Target and units need confirmation
Random Forest	76.31%	482.9094	Target and units need confirmation
XGBoost	78.15%	347.7295	Selected in paper; metric protocol must be verified

Table 6: Model performance values reported in the group research paper

6.2 Product and technical results

The documented product workflow includes METAR acquisition, parsing, Superbase storage, feature generation, XGBoost inference, a Flask API, and a cloud-and-visibility interface. These elements describe the intended implementation architecture in the group paper. The final individual thesis should add direct evidence from the source code, a running application, or test logs to demonstrate which parts were completed by the candidate and which were supplied by the shared system.

Technical results should include the parser's success rate on a representative sample, the number of valid records reaching each model, the API response format, response time under a defined test, and the number of test cases passed. None of these figures are present in the supplied source documents, so they are listed as evidence requirements rather than filled with estimates.

An implementation result is strongest when it is reproducible. A screenshot alone demonstrates a visible state but not necessarily correctness. Pair screenshots with test records, representative input/output examples, code references, and a description of the expected behaviour. The final document should avoid presenting a planned feature as a completed feature if it was not implemented.

6.3 User or stakeholder validation

The wider project identifies pilots, air traffic controllers, aviation meteorological officers, and airport operational managers as stakeholders. Their needs differ: a pilot-facing view must be concise and time-specific, while a meteorological reviewer may require more detail about the source observation, target horizon, and model confidence. The cloud-and-visibility output should therefore make its status and limitations visible without overloading the user with internal model details.

No stakeholder interview, usability test, or formal acceptance record was included in the supplied documents. Consequently, this thesis does not claim that pilots or meteorological officers validated the interface. If such a review occurred, the final version should document participant role (without unnecessary personal details), the tasks performed, the questions asked, the findings, and any limitations. If no such review occurred, the chapter should state that stakeholder validation remains future work.

Any future validation should avoid asking users only whether the interface looks clear. It should assess whether they can correctly distinguish observation from prediction, identify the target time, understand the predicted condition, recognise missing/stale data, and interpret the model output as supplementary information rather than operational clearance.

6.4 Integration results

Successful integration requires consistent timestamps and schemas between the data service, prediction API, and dashboard. The cloud-and-visibility outputs should not be merged with wind or pressure forecasts unless each value retains its own target time and model identity. A composite guidance card should clearly label the source and status of each field.

The group paper describes a broader system that combines weather predictions with aviation-relevant information and warning outputs. However, the supplied evidence does not include end-to-end test logs or an integration acceptance report for this individual component. Therefore, integration is described here as the documented

architecture and intended interface, while formal success claims await execution evidence.

Recommended evidence includes one complete trace showing a raw METAR report, the parsed record, the feature vector, model output, prediction interpretation, API response, dashboard presentation, and later verification against the target observation. This trace would help an examiner confirm that the prediction is produced and consumed consistently.

6.5 Summary of findings

The project paper identifies XGBoost as the selected algorithm for cloud and visibility after comparison with Decision Tree and Random Forest. It reports a top accuracy value of 78.15% for XGBoost, while the numerical error metric requires clarification. The paper also documents the data-to-model-to-API-to-dashboard workflow. These are the strongest source-supported results available for this draft.

7 DISCUSSION

7.1 Interpretation of findings

The reported XGBoost accuracy of 78.15% is higher than the Decision Tree and Random Forest accuracy values presented in the group paper. If the models were evaluated on the same test records using the same target and metric definition, the comparison supports selecting XGBoost among those three tested alternatives. However, the size and practical significance of the improvement cannot be established without sample counts, uncertainty estimates, and a clear description of the target.

The error values require more care. The group paper labels them as MAE but does not provide enough detail in the supplied extract to establish the units or whether the values relate to a scaled target, a transformed label, or a particular subset. Reporting these numbers without qualification could mislead a reader. The final analysis should retrieve the original code and evaluation output, confirm units, and recalculate metrics from the saved test predictions.

7.2 Comparison with literature and existing solutions

Prior Sri Lankan weather-prediction research establishes that data-driven approaches have been explored for short-term weather and rainfall prediction. Aviation-specific work establishes that visibility, fog, and cloud conditions matter to flight operations. The current component combines these motivations in an airport-level pipeline. It should not claim to outperform official products or published systems because the available evidence does not show a head-to-head comparison using a shared dataset.

Compared with a broad numerical weather prediction product, a local model can learn relationships from observations at a specific aerodrome. This can be advantageous for local calibration, but it also narrows the model's training distribution and may reduce portability. A model trained at VCBI should not be assumed to generalise to another airport without local data and validation.

Compared with a manually interpreted observation, a model can provide repeatable estimates and automate a prediction interpretation check. The trade-off is that model errors may be difficult to detect without clear uncertainty and source metadata. A

responsible system should preserve the official report, show the prediction separately, and provide a later verification record.

7.3 Research contribution

The contribution's strength depends on reproducible target definitions and evaluation. If the exact labels, data split, metrics, and test records are not available, the novelty claim should remain modest. A carefully documented local system with honest limitations is more defensible than a broad claim of improved aviation safety unsupported by evidence.

The individual contribution should also be distinguished from group-level contributions. Data acquisition, other weather models, overall architecture, flight-time processing, and document generation may be shared or assigned to other members. This thesis focuses on the cloud-and-visibility component and its integration points.

7.4 Product contribution

The product contribution is a component that can provide cloud-status and visibility estimates through a service boundary and make the outputs available for a dashboard. It can support a more complete weather picture when combined with the group's wind, temperature, pressure, and humidity outputs. Its user value comes from timely, legible, traceable information rather than from the model algorithm alone.

Product readiness includes security, reliability, monitoring, and documentation. Before use beyond an academic prototype, the component would need operational stakeholder review, appropriate verification, resilience testing, governance of model changes, and a clearly defined statement of intended use.

7.5 Limitations and threats to validity

Construct validity is threatened if the model's target does not represent the intended aviation concept. Visibility, runway visual range, cloud amount, cloud base, and ceiling are related but not interchangeable. The final thesis must document the target precisely and avoid interpreting one as another.

Internal validity is threatened by data leakage, duplicated observations, incorrect temporal alignment, and preprocessing performed before the train/test split. External

validity is limited by the focus on VCBI and the particular periods represented in the training data. A model may behave differently in another season, during rare events, or at another airport.

Metric validity is threatened by ambiguous reported error units and the absence of class-wise results in the available paper. Operational validity is limited because no formal certification, operational trial, or stakeholder acceptance evidence was supplied. Software validity is limited until test logs and code evidence establish that the implemented API and dashboard behave as intended.

7.6 Implications

For researchers, this component demonstrates the importance of separating the model, preprocessing, output interpretation logic, and verification procedure. For developers, it emphasises stable API contracts and shared transformations between training and inference. For aviation stakeholders, it illustrates how a supplementary prediction can be presented while preserving the authority of official weather products.

The work also suggests that rare-event evaluation should be prioritised. Average performance can conceal errors in fog or low-cloud conditions. Future evaluation should report recall, precision, false-alarm rate, lead time, and error severity for the conditions that matter most to the intended user.

Any wider adoption should be staged. First, establish reproducible offline results. Second, conduct shadow-mode evaluation in which predictions are recorded but do not influence operational decisions. Third, obtain expert review and assess failure modes. Only after governance, validation, and approval should any operational role be considered, and the model should never be represented as certified solely because it is integrated into a dashboard.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

The supplied group paper describes separate XGBoost models for cloud status and visibility, feature engineering based on METAR-derived variables, a Flask API, Superbase storage, and an interface for presenting outputs. It reports 78.15% accuracy for XGBoost in its comparison with Decision Tree and Random Forest. However, the exact target definitions, units for the reported error values, component-specific sample count, test-period details, and low-visibility recall were not supplied. Those limitations prevent a stronger claim about operational reliability.

The principal conclusion is that the described design is a plausible applied approach to local prediction and integration, but the final academic claim must be proportional to the evidence. The model should be evaluated with clear target definitions, chronological test data, relevant baselines, and metrics that specifically measure rare adverse conditions. It should remain supplementary to official aviation-weather products.

8.2 Achievement of objectives

The design objective is supported by the documented workflow that links METAR input, parsing, feature engineering, model inference, output interpretation logic, and presentation. The model-selection objective is partially supported by the reported comparison favouring XGBoost. The integration objective is described in the group paper but requires source-code and end-to-end test evidence to demonstrate implementation completion.

A final objective matrix should include a status, evidence artefact, and citation for each objective. If an objective was not completed, it should be described honestly as partially achieved or future work rather than being silently treated as complete.

8.3 Individual research and product contributions

Group infrastructure is acknowledged separately. Shared data acquisition, database hosting, other prediction components, the overall dashboard, and system-wide verification should not be claimed as individual work unless the candidate can

demonstrate direct ownership. A contribution table and version-control evidence will make the boundary clear to examiners.

The strongest evidence package would contain a repository snapshot, model artefacts, feature list, data dictionary, training/evaluation script, test output, API request/response examples, and screenshots of the running component. These artefacts would convert the architecture description into an auditable account of implementation.

8.4 Future directions

The immediate priority is to resolve the metric ambiguity by inspecting the original experiment code and confirming the target variable, units, and evaluation procedure. Then calculate a complete set of target-specific results on a chronologically separated test set. For cloud status, include per-class metrics and a confusion matrix. For visibility, report regression metrics in the target unit or classification metrics if visibility was categorised. Compare against a simple baseline.

The second priority is to evaluate difficult meteorological cases explicitly. A documented subset of fog, low-cloud, and rapid-visibility-change observations should be identified using the source report and an agreed labelling rule. The study should report the number of cases, correctly detected cases, missed cases, and false positives, together with the target definition and evaluation window. Errors near rapid transitions deserve separate inspection because averages across all observations may conceal weaknesses on uncommon but important cases. If class imbalance is present, class weighting or resampling may be considered within training data only, with the final test set left untouched.

The third priority is operational hardening: record model versions and timestamps, add data-quality and stale-input warnings, validate API contracts, monitor model drift, and define a review process for retraining. Future work may also explore probabilistic outputs, calibrated confidence estimates, sequence-based architectures, and transfer to other airports, but each extension should be justified by a clear evaluation question rather than added for novelty alone.

REFERENCES

REFERENCES

[1] International Civil Aviation Organization, "Meteorological Service for International Air Navigation," International Civil Aviation Organization (ICAO), 2018.

[2] Federal Aviation Administration, "Pilot's Handbook of Aeronautical Knowledge," U.S. Department of Transportation, Federal Aviation Administration, 2023.

[3] S. Asifa, B. M. Samarsitha, R. Neha, R. N. Jalagam and G. Saranya, "Lightning Risk Prediction for Aviation Safety," in *2025 1st International Conference on Radio Frequency Communication and Networks (RFCoN)*, 2025.

[4] P. Boonyuen, F. Wu, P. Phunthirawuth** and Y. Zhao, "Impact of GPS RO and Radiance Data Assimilation on Numerical Weather Prediction," in *2016 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, Beijing, China, 2016.

[5] P. D. Dueben and P. Bauer, "Challenges and design choices for global weather and climate models based on machine learning," *Geoscientific Model Development,* vol. 11, no. 10, pp. 10.5194/gmd-11-3999-2018, 2018.

[6] P. Hu, J. Zhang and N. Li, "Research on Flight Delay Prediction Based on Random Forest," in *2021 IEEE 3rd International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, Changsha, China, 2021.

[7] S. Rasp and S. Lerch, "Neural Networks for Postprocessing Ensemble Weather Forecasts," *Monthly Weather Review,* vol. 146, no. 11, p. 3885–3900, 2018.

[8] S. K. Sharma, D. K. Pandiya, V. R. Joshi, P. Bansal and D. K. Sharma, "Artificial Intelligence for Real-Time Weather Forecasting Using Neural Networks," in *Intelligent Systems Using Semiconductors for Robotics and IoT*, CRC Press, 2025, p. 347–353.

[9] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-k. Wong and W.-c. Woo, "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montreal, Canada, 2015.

GLOSSARY

Term	Definition
Aerodrome	An area intended for the arrival, departure, and surface movement of aircraft.
Cloud ceiling	The lowest cloud layer meeting the applicable broken/overcast or vertical-visibility reporting definition; use the standard applicable to the dataset.
Feature engineering	Creating or transforming input variables to represent useful patterns for a predictive model.
Forecast issue time	The time at which a prediction is generated.
Forecast target time	The future time or period that the prediction is intended to describe.
Leakage	Use of information during training or feature preparation that would not legitimately be available at prediction time, or contamination of evaluation data.
METAR	A coded routine aerodrome weather observation.
Model drift	Change in the relationship between inputs and targets or in the input distribution that can reduce predictive performance.

Present weather	A METAR field describing current weather phenomena using coded values.
Visibility	A reported measure of visual range under the relevant meteorological reporting convention.
XGBoost	An implementation of gradient-boosted decision trees commonly used for structured/tabular prediction.

APPENDICES

Appendix A. Extended technical details and workflow

The following workflow is a design-level representation based on the supplied group paper. Replace or supplement it with the exact module names and function names from the repository before submission.

Stage	Input	Operation	Output / check
1. Acquisition	METAR report from source API	Retrieve report and timestamp it	Raw report retained
2. Parsing	Raw METAR text	Extract supported fields and handle special encodings	Structured record plus parse status
3. Validation	Structured record	Check types, ranges, timestamps and required fields	Valid record or explicit rejection
4. Feature generation	Validated record and allowed history	Apply documented transformations and interactions	Feature vector with expected schema
5. Inference	Feature vector and model artefacts	Load XGBoost cloud and visibility models	Predicted target values/classes
6. Threshold evaluation	Prediction and approved configuration	Apply boundary rule	Warning status and reason
7. API response	Predictions and metadata	Serialize result for client	Consistent response contract

8. Presentation	API response	Display predictions and freshness information	Dashboard card/visual
9. Verification	Prediction and later target observation	Match timestamps and calculate errors	Verification record

Table 7: Data-to-prediction processing stages

A.1 Feature documentation checklist

Field	Question to resolve before submission
Visibility target	Is it continuous metres, coded visibility, or a categorical class?
Cloud target	Is the target cloud amount, cloud status, ceiling height, or a hazard label?
Forecast horizon	Is the prediction for a fixed horizon, next report, or flight-time block?
Timestamp	Is the source timestamp UTC? How is the target observation matched?
Missing values	Which values are missing, invalid, or meaningful special codes?
Present weather	How are fog, mist, rain and unknown codes encoded?
Cloud layers	How are multiple layers and vertical visibility represented?
Scaling	Is any normalisation used, and is it saved with the model?

Categorical features	How are unseen categories handled?
Model artefact	What file, version, date and training configuration are used?

Appendix A.2 Data quality and preprocessing protocol

A.2.1 Raw-data preservation

The raw METAR string should be retained alongside its parsed representation whenever data governance rules permit. Keeping the original report makes it possible to inspect whether an unusual model output came from an atmospheric pattern, a parser defect, or a missing field. The raw report should not be overwritten by a normalised representation. A stable record identifier, source timestamp, retrieval timestamp, parser version, and parse status make the record traceable across the pipeline. If the system stores only parsed fields, it should document how the original report can be recovered and what happens when that is not possible.

A.2.2 Timestamp normalisation

Time handling is central to forecast evaluation. The source report may include a timestamp convention that differs from the application server's local timezone. The pipeline should convert timestamps to a consistent internal representation, usually UTC, while retaining the source value for audit. Daylight-saving assumptions should not be applied to UTC records. When a prediction is issued, the issue timestamp and target timestamp must be distinct fields. A test that compares a prediction with the nearest available METAR without a stated tolerance can accidentally compare it with an observation from the wrong period.

A.2.3 Duplicate observations

Duplicate records may arise from repeated API polling, data refreshes, or ingestion retries. Deduplication should use a documented key, such as a source report identifier where available or a combination of station, observation timestamp, and raw report. It should not remove legitimate reports merely because two consecutive reports have similar values. The pipeline should log the number of duplicates detected and removed, and it should make the rule reproducible. Duplicate removal must occur before data splitting so that the same observation cannot appear in both training and test data.

A.2.4 Missing and invalid values

Missing values require field-specific interpretation. A missing visibility value, an absent cloud group, an unknown weather code, and an explicitly reported clear condition are not equivalent. Replacing every missing value with zero can create a false signal, particularly when zero visibility has a severe meteorological meaning.

The parser should distinguish not reported, not applicable, invalid, and explicit coded conditions wherever the source format allows. Imputation must be justified and fitted using training data when the imputation parameters are learned from the data.

A.2.5 Outlier handling

Outlier review should begin with domain and encoding checks rather than automatic removal. A very low visibility observation may be a valid fog event, not a data error. Similarly, a sudden change can be exactly the event the model is intended to detect. Values should be flagged when they violate documented physical or encoding constraints, but the raw value should remain available for audit. Any filtering rule should be stated, applied consistently, and reported with before-and-after counts. The test set should not be selectively cleaned using knowledge of its target outcomes.

Appendix A.3 METAR parsing and target-definition considerations

A.3.1 Parsing is part of the model

A predictive model receives structured values, not the full operational meaning of an aviation report unless that meaning is encoded deliberately. A parser defect can produce a plausible numeric value with the wrong unit or category. For that reason, the parser must be treated as a component requiring tests and versioning, rather than as an unimportant pre-processing step. The model's accuracy is conditional on the correctness of its input representation.

A.3.2 Visibility target representation

Before evaluating visibility, the implementation must state whether the target is a continuous measurement, a category, or a threshold label. If continuous, the unit and any transformation must be recorded. If categorical, the class boundaries and their source must be specified. If a threshold label is used, the positive class must be explicitly defined. These formulations answer different questions and require different metrics. A value that represents a visibility code should not be interpreted as metres unless a verified conversion has been applied.

A.3.3 Cloud target representation

Cloud-related targets can represent amount, layer height, ceiling, vertical visibility, or a derived adverse-condition class. The project paper uses the phrase cloud status but the supplied documents do not fully define the final target schema. The final thesis must resolve this ambiguity using the training script and saved labels. When multiple cloud layers exist, the rule for selecting or representing the relevant layer must be documented. Any conversion from coded layer groups to a ceiling estimate should be verified against the applicable reporting convention.

A.3.4 Present-weather encoding

Present-weather codes can carry useful context for visibility reduction, but a model must not use a future weather code to predict an earlier target. The feature should be taken from the observation available at the prediction issue time. Categories should be encoded consistently; rare or unseen codes need a defined handling strategy. A category mapping should be stored with the model artefact so the same encoding can be reconstructed after retraining.

A.3.5 Target leakage examples

Leakage can occur if the target visibility value is accidentally included in the input feature vector, if a feature is calculated using the future report, or if a data split allows near-duplicate future observations to appear in training. It can also occur when preprocessing statistics are computed on the full dataset before splitting. Leakage can create unexpectedly strong scores without a genuinely predictive model. A feature audit should record the source timestamp for each variable and verify that it is available at inference time.

Appendix A.4 Model training and validation protocol

A.4.1 Establishing a baseline

A baseline is necessary to show whether the trained model adds useful predictive information. For a continuous visibility target, persistence predicts that the next value will equal the latest available value; a training-mean predictor can also be reported as a simple reference. For a categorical cloud target, persistence or the majority class provides a reference. The selected baseline should be appropriate to the prediction horizon and should use the same test records as the XGBoost model.

A.4.2 Chronological partitioning

A chronological split preserves the direction of time. The training period should precede the validation period, which should precede the final test period. Exact boundaries should be recorded in the thesis. If the model is tuned using cross-validation, the folds should respect temporal ordering. A final test period should remain untouched until the modelling choices are fixed. If records are clustered by reporting interval, the split should also prevent the same target observation from being duplicated across partitions.

A.4.3 Hyperparameter selection

Hyperparameters can affect tree depth, learning rate, number of boosting rounds, regularisation, and sampling. The selection process should be described with the search space, validation method, scoring metric, and chosen values. A large search on the final test set invalidates that set as an unbiased evaluation. The final document should report the actual values from the saved model configuration; it should not invent a standard configuration merely because it is common in examples.

A.4.4 Class imbalance

Low visibility and fog may occur less frequently than ordinary conditions. A model can therefore appear accurate while rarely identifying the event of interest. Class counts and per-class metrics are required to understand this problem. If class weighting or resampling is used, it must be applied within the training process only. Resampling before the split can leak near-duplicate cases into evaluation and distort the apparent event frequency.

A.4.5 Repeated runs and uncertainty

A single score does not communicate how stable performance is. Where training randomness is involved, repeated runs or confidence intervals can help describe variation. For time-dependent data, uncertainty estimation should respect temporal structure rather than assume every observation is independent. The final thesis should state whether the reported number comes from one fixed model, an average over repeated runs, or a selected best run.

Appendix A.5 Interpreting cloud and visibility metrics

A.5.1 Accuracy and its limits

Accuracy is the proportion of classification predictions that match the reference labels. It is meaningful only when the target is categorical and the label mapping is defined. In imbalanced data, accuracy can be high even when a rare but important class is not detected. The reported 78.15% figure in the supplied paper should therefore be linked to the exact model target and test set before it is used as a headline result.

A.5.2 Mean absolute error

MAE is the mean absolute difference between predicted and observed numeric values. It has the same unit as the target, which makes unit reporting essential. An MAE of 10 metres, 10 kilometres, or 10 encoded category units would have very different meanings. The group paper's reported error values require confirmation of the target and scaling before an operational interpretation is possible.

A.5.3 Root mean squared error

RMSE is the square root of the average squared error. Because the errors are squared before averaging, a small number of large misses can increase RMSE substantially. Reporting both MAE and RMSE can help distinguish typical error from sensitivity to larger errors. For visibility, the thesis should include a residual distribution and examples of large underestimates and overestimates.

A.5.5 Confusion matrix and event-level analysis

A confusion matrix shows which classes are confused. For example, it can reveal whether a model systematically classifies low cloud as a less adverse category. Event-level analysis should inspect the timing of errors around transitions, not only individual rows. Repeated predictions during one event should not automatically be counted as independent successful detections without stating the evaluation method.

Metric	Target type	What it tells the reader	Common caveat
Accuracy	Classification	Overall correct fraction	Can hide rare-class failures

Precision	Classification / alert	Trustworthiness of positive predictions	Depends on event prevalence
Recall	Classification / alert	Fraction of events detected	May rise as false alarms rise
Macro-F1	Multi-class classification	Balances class-level precision and recall	Does not express physical-unit error
MAE	Regression	Average absolute error in target units	Units and scale must be stated
RMSE	Regression	Penalises large errors more strongly	Sensitive to outliers

Table 8: Evaluation metrics and their limitations

Appendix A.6 API contract and error-handling specification

A.6.1 Request contract

A request contract should state the required fields, optional fields, types, units, timestamp format, and accepted categorical values. If the API receives a raw METAR report, it should document whether parsing happens inside the endpoint or upstream. If it receives an already parsed object, it should identify the schema version. A clear contract allows model testing independently of the front-end application.

A.6.2 Response contract

A successful response should identify the request or observation, issue time, target time, model version, cloud output, visibility output, warning status, and data-quality status. If confidence values are not actually calculated, they should not be fabricated. A model probability, if available, should be labelled according to its calibration status. Units should be returned with numeric values or unambiguously defined in the schema.

A.6.3 Error responses

Expected errors include malformed input, missing required features, unavailable model artefacts, database timeouts, unsupported categories, and internal prediction failures. The API should return an appropriate status and a concise machine-readable error code. It should not return a default weather value after an exception, because a plausible-looking default can be mistaken for a real prediction.

A.6.4 Observability

Logs should record enough information to reconstruct the processing path without storing unnecessary sensitive data. Useful fields include request identifier, source timestamp, parser status, model version, processing duration, and error category. Logs should avoid exposing API keys or database credentials. A health endpoint can report whether the service and model artefacts are available, but it should not expose internal configuration secrets.

A.6.5 Versioning

Changes to the input schema, target definition, model artefact, or target interpretation rule can change the meaning of a prediction. These changes should be versioned and documented. A client should be able to determine which model produced a result.

When a new version is deployed, a smoke test should confirm schema compatibility and a rollback plan should be available.

Appendix A.7 Dashboard presentation and accessibility

A.7.1 Distinguishing observation and prediction

The dashboard should use explicit labels such as “Observed METAR” and “Model prediction” rather than relying on different colours or positions alone. Each value should include its timestamp. If the observed visibility is from the present report and the prediction refers to a later target time, that distinction should remain visible even on small screens or compact cards.

A.7.2 Displaying units and rounding

A number without a unit is ambiguous. Visibility should use the unit associated with the model's target representation, and any conversion should be documented. Rounding should not change the prediction interpretation. If a predicted value is rounded for display, the underlying decision should use the unrounded prediction, and the interface should avoid implying more precision than the model supports.

A.7.4 Accessibility

Colour should not be the only carrier of meaning. Pair warning colours with text, icons or labels, and ensure readable contrast. Keyboard navigation and screen-reader semantics are relevant if the interface is web-based. These requirements improve usability in general and reduce the chance that a critical distinction is missed under time pressure.

A.7.5 User evaluation

A structured review should ask participants to interpret example cards, identify the forecast target time, distinguish the official observation from the model prediction, explain the meaning of the prediction, and identify what to do when the data are stale. The review should record task completion and misunderstanding, not just subjective preference. No such validation is claimed in this draft because no results were supplied.

Appendix A.8 Risk register

Risk	Potential effect	Mitigation	Evidence / owner to confirm
Incorrect METAR parsing	Invalid features and misleading predictions	Parser tests and raw-record audit	Implementation owner to confirm
Target ambiguity	Metrics do not measure intended outcome	Define label, unit and horizon	Individual component owner
Temporal leakage	Overstated test performance	Chronological split and timestamp audit	Model evaluation owner
Rare adverse events	Low recall for fog/low cloud	Subgroup metrics and event review	Model evaluation owner
Unverified threshold	Incorrect warning status	Authoritative source and versioned config	Domain supervisor / owner
Stale source data	Old output appears current	Freshness check and explicit label	Backend owner
Model drift	Performance degrades over time	Monitoring and scheduled re-evaluation	MLOps owner
API or storage outage	Predictions unavailable or unlogged	Health checks and controlled errors	Service owner

Credential exposure	Unauthorised data access	Secret management and access controls	Repository maintainer
Overreliance on prediction	Users mistake output for operational approval	Clear intended-use statement and review	Project team
Unclear personal contribution	Academic integrity concern	Code ownership and contribution matrix	Candidate
Unverified references	Citation inaccuracies	Check original publications	Candidate

Table 9: Risk register for the prediction component

The register is a planning artefact. Assign a named owner only after confirming the project's actual responsibilities. A risk should be considered closed only when the mitigation is implemented and supported by evidence.

Appendix A.9 Requirements traceability matrix

Objective / requirement	Design location	Implementation evidence	Evaluation evidence
Identify METAR input fields	Sections 3.2 and 4.4	Parser and data dictionary	Parser unit tests
Generate consistent features	Sections 3.3 and 4.4	Feature builder code	Feature parity tests
Predict cloud status	Sections 4.4 and 5.2	Cloud model artefact and wrapper	Target-specific test metrics
Predict visibility	Sections 4.4 and 5.2	Visibility model artefact and wrapper	Error metrics with units or class metrics
Integrate through API	Sections 4.2 and 5.3	Endpoint and schema	API/integration test logs
Present output to user	Sections 4.2 and 5.3	Dashboard component	Screenshot and usability evidence
Verify predictions later	Sections 3.4 and 5.4	Forecast/observation matching code	Verification log and metrics
Maintain model responsibly	Sections 3.6 and 5.5	Versioned artefacts and monitoring	Retraining/evaluation records

Table 10: Requirements traceability matrix

This matrix should be updated to refer to actual filenames, functions, test identifiers, and commit hashes. It is intentionally not marked complete because the supplied files do not include the implementation repository or executed test artefacts.

Appendix A.10 Literature comparison matrix

Source / topic	Problem domain	Relevance	Limit when compared with this component
Sri Lankan ANN monsoon rainfall study	Rainfall forecasting	Local use of ML for meteorology	Rainfall is not airport visibility
Sri Lankan short-term weather ML study	General short-term weather	Regional research context	Target and spatial scale may differ
High-impact aviation weather review	Aviation hazards	Motivates reliable local weather information	Review is not an evaluation of this model
Automated TAF generation study	Aviation forecast automation	Aviation-specific ML context	May predict different targets/horizons
ConvLSTM nowcasting study	Precipitation nowcasting	Shows sequence-based deep learning potential	Different input structure and target
Global ML weather modelling discussion	Large-scale weather models	Context for ML-based prediction limits	Not a VCBI METAR-only comparison
Neural network ensemble postprocessing	Forecast postprocessing	Context for statistical correction	Different model inputs and verification design

Current component paper	VCBI cloud and visibility	Direct project evidence for selected workflow	Metrics and target schema need clarification

Table 11: Literature comparison matrix

This matrix is a synthesis of the sources listed in the supplied group paper. Before submission, add full verified citations and extract the precise dataset, method, horizon and metric from each original publication. Do not infer details from titles alone.

Appendix A.10 Data governance and model card

A.10.1 Intended use

The intended academic use is to investigate short-term cloud and visibility prediction at VCBI from METAR-derived data and to demonstrate how model outputs can be integrated into a supplementary weather-awareness system. The component is not represented as a certified aviation decision system and must not be used as the sole basis for a flight or runway decision.

A.10.2 Out-of-scope use

The model should not be assumed to generalise to airports, climate regimes, sensors, reporting standards, or forecast horizons that are not represented in its evaluation. It should not be used to replace official observations, official forecasts, approved minima, or professional judgement. The exact scope should be revised if the actual training data and validation demonstrate a broader supported domain.

A.10.3 Data description

The project paper describes historical VCBI meteorological observations from 2019–2026 and live METAR data retrieved from the Aviation Weather Center. The precise component-specific sample count, missingness, target distribution, and split dates must be added from the implementation. Any restrictions on raw-data redistribution should be documented.

A.10.4 Model details

The group paper identifies XGBoost as the selected method for cloud status and visibility after comparing it with Decision Tree and Random Forest. The actual XGBoost configuration, feature list, target type, training date, package version, and artefact checksum remain to be inserted from the model files.

A.10.5 Performance and limitations

The supplied paper reports 78.15% accuracy and an error value of 347.7295 for XGBoost in the cloud-and-visibility comparison. The error's unit and target representation require confirmation. No class-specific adverse-weather metrics are supplied, so the model card must not claim reliable detection of every fog or low-cloud event.

A.10.6 Maintenance

A model update should be accompanied by a new version identifier, data-window description, evaluation report, and approval record. Performance should be monitored as future observations arrive. If the model's performance degrades or the input schema changes, the system should fall back to a documented unavailable state rather than continue presenting stale results as current.

Model-card field	Current draft entry
Model name	Cloud and Visibility Prediction Model
Algorithm	XGBoost, as described in the supplied group paper
Geographic context	VCBI / Bandaranaike International Airport
Input source	METAR-derived observations
Targets	Cloud status and visibility; exact schema to confirm
Reported comparison	XGBoost accuracy reported as 78.15%; error metric needs clarification
Intended use	Supplementary academic prediction and evaluation
Known limitations	Target/metric ambiguity, missing subgroup metrics, no supplied operational validation
Version / training date	To be completed from artefacts

Table 12: Model-card fields and current evidence status

Appendix B. Test cases and evidence checklist

The following test cases are proposed verification activities, not claims that the tests have already passed. Record actual input, expected result, observed result, status, date, and software/model version when executing them.

Test ID	Test scenario	Expected outcome
B01	Parse valid METAR with visibility and cloud layers	All supported fields extracted; raw report retained
B02	Parse report with fog present-weather code	Fog code mapped according to documented dictionary
B03	Parse report with no cloud layer reported	Interpretation follows METAR semantics, not a generic null rule
B04	Parse malformed cloud-height token	Controlled parse error or documented recovery
B05	Missing visibility field	No silent conversion to zero visibility
B06	Negative or impossible numeric input	Rejected or flagged by validation
B07	Unseen present-weather category	Handled consistently with training schema
B08	Input timestamp in wrong format	Validation error with clear message
B09	Duplicate observation	Deduplication rule applied and logged

B10	Feature builder output	Names, order, types and values match training contract
B11	Cloud model inference	Returns expected target type and metadata
B12	Visibility model inference	Returns value/class with documented unit/encoding
B13	Model artefact missing	Service reports controlled failure
B17	Stale observation	Output marked stale or rejected
B18	Storage unavailable	Error logged; no fabricated success response
B19	API response schema	Required fields present and serializable
B20	Dashboard integration	Predicted and observed values are visually distinct
B21	Future METAR becomes available	Forecast is matched to the correct target timestamp
B22	Model update	New version is recorded and smoke-tested
B23	Low-visibility subset evaluation	Recall, precision and confusion matrix calculated
B24	Ordinary-condition subset evaluation	False alerts and error distribution reported

B25	Repeat inference with same input/model	Output is reproducible within expected numerical tolerance

Table 13: Extended parser and integration test cases

Appendix B.2 Academic integrity and AI-use disclosure

This section must be completed according to the current IT4010/module policy. The candidate should list every generative AI or automated tool used, the purpose of use, representative prompts or workflows where required, the verification performed, and the sections or artefacts affected. The use of assistance to draft or format this document does not remove the candidate's responsibility for checking technical accuracy, confirming citations, validating the results, and ensuring that the final work represents their own research and contribution.

Suggested disclosure fields:

Complete truthfully according to the tools actually used and the institution's academic-integrity policy.

REFERENCES

[1] International Civil Aviation Organization, "Meteorological Service for International Air Navigation," International Civil Aviation Organization (ICAO), 2018.

[2] Federal Aviation Administration, "Pilot's Handbook of Aeronautical Knowledge," U.S. Department of Transportation, Federal Aviation Administration, 2023.

[3] S. Asifa, B. M. Samarsitha, R. Neha, R. N. Jalagam and G. Saranya, "Lightning Risk Prediction for Aviation Safety," in *2025 1st International Conference on Radio Frequency Communication and Networks (RFCoN)*, 2025.

[4] P. Boonyuen, F. Wu, P. Phunthirawuth** and Y. Zhao, "Impact of GPS RO and Radiance Data Assimilation on Numerical Weather Prediction," in *2016 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, Beijing, China, 2016.

[5] P. D. Dueben and P. Bauer, "Challenges and design choices for global weather and climate models based on machine learning," *Geoscientific Model Development,* vol. 11, no. 10, pp. 10.5194/gmd-11-3999-2018, 2018.

[6] P. Hu, J. Zhang and N. Li, "Research on Flight Delay Prediction Based on Random Forest," in *2021 IEEE 3rd International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, Changsha, China, 2021.

[7] S. Rasp and S. Lerch, "Neural Networks for Postprocessing Ensemble Weather Forecasts," *Monthly Weather Review,* vol. 146, no. 11, p. 3885–3900, 2018.

[8] S. K. Sharma, D. K. Pandiya, V. R. Joshi, P. Bansal and D. K. Sharma, "Artificial Intelligence for Real-Time Weather Forecasting Using Neural Networks," in *Intelligent Systems Using Semiconductors for Robotics and IoT*, CRC Press, 2025, p. 347–353.

[9] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-k. Wong and W.-c. Woo, "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montreal, Canada, 2015.